KHÓA LUẬN TỐT NGHIỆP · LUẬN CỨ ĐỀ TÀI

Vì sao đề tài này đáng làm

Tầm quan trọng, khoảng trống nghiên cứu, và tính hữu ích của một oracle phân quyền có trạng thái/thời gian cho bộ fuzzer REST API

Nguyễn Minh Mạnh · Ngày 31/07/2026 · Mọi nguồn học thuật đã được xác minh first-party (xem §6)

Tóm tắt. Đề tài đáng làm dựa trên ba chân kiềng, mỗi chân có bằng chứng độc lập: **(1) Tầm quan trọng** — phân quyền là lớp lỗ hổng API nghiêm trọng nhất theo OWASP, và tiểu lớp *có trạng thái* (quyền còn sống sau khi lẽ ra đã bị thu hồi) là nguy hiểm và ít được kiểm nhất. **(2) Khoảng trống** — không công cụ nào đã công bố kiểm tra sự phân quyền *xuyên qua một bước làm giảm quyền* bằng cách phát lại token cũ; các công cụ mạnh nhất đều so sánh *khác người, một thời điểm*. **(3) Tính hữu ích đã chứng minh** — không phải hứa hẹn: công cụ SOTA (EvoMaster + oracle baseline) đã được tái lập và cho thấy *mù về mặt cấu trúc* với lỗi này, trong khi oracle của luận văn phát hiện đúng nó với **precision hoàn hảo** qua 40 lần chạy có kiểm soát.

1. Vấn đề và vì sao nó quan trọng

Phân quyền — *ai được phép làm gì* — là lớp lỗ hổng API **nghiêm trọng nhất trong thực tế**. Trong **OWASP API Security Top 10 (2023)**, hai trong ba hạng mục đầu bảng là lỗi phân quyền: **API1 — Broken Object Level Authorization (BOLA)** và **API5 — Broken Function Level Authorization (BFLA)** [7]. Ở bảng OWASP Top 10 cho ứng dụng web, *Broken Access Control* đứng **hạng nhất**; một công trình bảo mật gần đây (BACScan, CCS 2025 [4]) dẫn lại rằng trong OWASP 2023, lỗi kiểm soát truy cập chiếm **bốn trong năm** lỗ hổng nghiêm trọng nhất.

Trong nhóm phân quyền, dạng nguy hiểm và khó phát hiện nhất là dạng **có trạng thái (stateful)**: một người dùng vẫn *giữ được quyền truy cập sau khi* một thay đổi lẽ ra đã thu hồi quyền đó. Ví dụ điển hình (lớp mục tiêu của luận văn):

> **KỊCH BẢN LỖI (A1)**
> Một quản trị viên (ADMIN) tạo `/reports/9`, rồi **bị hạ quyền xuống VIEWER**. Nhưng **chính token cũ** mà họ đang cầm vẫn `DELETE /reports/9` thành công — đáng lẽ phải bị `403`. Việc thu hồi quyền chưa bao giờ được thực thi trên token đã cấp trước đó.

Vì sao lớp lỗi này bị bỏ lọt: nó **vô hình với một phép kiểm tra đơn yêu cầu**. Một phản hồi `200/204` sau khi hạ quyền trông y hệt một phản hồi hợp lệ. Lỗi chỉ lộ ra khi ta quan sát *một chuỗi* chứa bước làm giảm quyền, và phát lại *đúng credential cũ*. Đây không phải tình huống giả định: các hướng dẫn bảo mật thực hành mô tả đúng mẫu này ("demoted admin remains admin until cache refresh"; "tests that always log in after a role change cannot detect stale token claims") — *tuy nhiên đây là nguồn hành nghề, không phải học thuật, và chỉ dùng để minh họa rằng lỗi có thật ngoài đời.*

2. Khoảng trống: chưa ai lấp (đã xác minh first-party)

Không công trình nào đã công bố — học thuật hay thương mại — chiếm đúng ngách: *một oracle phân quyền theo hướng biến hình, theo thời gian, phát hiện truy cập còn tồn tại SAU một chuyển tiếp làm giảm quyền trong cùng một phiên của một người dùng, bên trong một fuzzer REST API*. Các "láng giềng" gần nhất, mỗi cái hụt trên một trục gọi tên được:

Công trình	Gần ở điểm	Hụt trên trục nào
Şahin, Zhang & Arcuri (2026) [2] — đường cơ sở trực tiếp	9 oracle bảo mật trong chính EvoMaster	**khác-người / một-thời-điểm**; không oracle nào mô hình hóa một chuyển tiếp giảm quyền theo thời gian
BACScan (Liu et al., CCS 2025) [4]	đối thủ kiểm soát truy cập mạnh nhất; 89 lỗ hổng / 35 CVE	nhắm vào **ứng dụng web** (không REST/OpenAPI); vẫn khác-người / một-thời-điểm
ARMeta (arXiv:2605.28321) [5]	gần nhất về *phương pháp* (metamorphic + REST)	mọi quan hệ biến hình đều thuần **chức năng**; dùng microservice RBAC làm đối tượng thử nhưng **không có quan hệ nào về phân quyền**
BACFuzz (arXiv:2507.15984, 2025)	fuzzer BAC grey-box cho **ứng dụng web PHP** (không REST/OpenAPI)	kiểm BOLA/BFLA bằng tài khoản **vai trò thấp** — khác-người / một-thời-điểm; không phát lại token cũ
AuthREST (arXiv:2509.10320, 2025)	kiểm thử tự động web API	nhắm vào **xác thực (authentication)**, không phải phân quyền

Sợi chỉ chung: không công cụ nào *phát lại một token cấp trước lúc hạ quyền, xuyên qua một bước giảm quyền, rồi kiểm xem hành động có còn được phép không*. Đó chính là "nước đi" của luận văn.

Trung thực về đường cơ sở: bài Şahin và cộng sự *không* tuyên bố khoảng trống này là hướng tương lai của họ — phần "future work" của họ chỉ nêu chung chung ("thiết kế thêm các oracle cho những loại lỗi bảo mật khác"). Điều đáng chú ý hơn: ngay trong thực nghiệm (Mục 5.4 của họ), họ *gặp* đúng các endpoint chỉnh sửa vai trò/đăng nhập người dùng nhưng chỉ coi đó là **một trở ngại kỹ thuật** (làm token đầu phiên hết hiệu lực), *không* biến nó thành phép thử phân quyền theo thời gian. Khoảng trống vì thế được lập luận **theo cấu trúc**, không dựa vào bất kỳ trích dẫn "họ tự nêu" nào.

3. Vì sao đây là đóng góp có ích — không phải "chỉ vài dòng code"

Một câu hỏi phản biện chính đáng: nếu là vấn đề lớn, sao có thể giải bằng ít mã như vậy? Ba điểm trả lời:

- **Đóng góp nằm ở *insight*, không ở số dòng code.** Phần mở rộng nhỏ (+321 dòng / 3 tệp) *chính vì* nó đứng trên hạ tầng khổng lồ của EvoMaster (phân tích schema, sinh chuỗi, quản lý token, xuất test). "Delta nhỏ khi xây trên nền có sẵn" là dấu hiệu kỹ thuật tốt, không phải nông.
- **Phần trí tuệ cốt lõi là hình thức hóa quan hệ.** Oracle được định nghĩa là một phép kiểm *"thu hồi có điều kiện"* — kiểm tra **sự thực thi (enforcement)** chứ không phải tính đơn điệu, với một "điều kiện canh" load-bearing (hành động phải *đòi hỏi* đúng quyền bị thu hồi). Đây là điều chưa ai viết ra, và là phần khó nhất — không phải đoạn code.
- **Đây là bản chất của đóng góp cấp-luận-văn trong ngành kiểm thử.** Bản thân *kiểm thử biến hình* (Chen, 1998) chỉ là ý tưởng "liên hệ hai lần chạy" — phát biểu thì tầm thường, tác động thì lớn. Cái khó là *nhìn ra*

một quan hệ chưa ai viết. Quan hệ ở đây (phát lại token cũ qua một bước hạ quyền) đúng kiểu đó: hiển nhiên khi nhìn lại, nhưng chưa ai viết ra.

> **PHẠM VI TRUNG THỰC**
> Luận văn **không** tuyên bố giải quyết toàn bộ lớp lỗi phân quyền. Nó chỉ ra một quan hệ biến hình *cụ thể* chưa ai hình thức hóa, chứng minh fuzzer SOTA mù với nó, và cài + đo được rằng oracle bắt đúng với precision hoàn hảo. Một đóng góp *có phạm vi rõ, kiểm chứng được, lấp một gap gọi tên được* là đúng tầm một khóa luận tốt nghiệp — mục tiêu "đạt một cách chắc chắn", không phải "xuất sắc mạo hiểm".

4. Bằng chứng đã có (không phải hứa hẹn)

4.1 Công cụ SOTA mù với lỗi này — đã tái lập

Chạy EvoMaster v6.1.1 với các oracle bảo mật baseline bật đầy đủ trên ứng dụng mục tiêu có kiểm soát, nó **không** phát hiện lỗi A1. Sự mù này mang tính **cấu trúc**, không phải may rủi, vì hai lý do độc lập: (i) *không phán xét được* — cả 9 oracle của baseline đều single-instant, không luật nào nói "quyền đã giảm ⇒ phải bị từ chối"; (ii) *không dựng được* — nó không bao giờ ghép "hạ quyền nạn nhân" với "nạn nhân thử lại" trong một chuỗi, và luôn đăng nhập lại nên không bao giờ phát lại token cũ.

4.2 Oracle của luận văn bắt đúng — precision hoàn hảo

Trên tầng thực nghiệm có kiểm soát (Tier-1), **40 lần chạy** (20 trên bản dính lỗi, 20 trên bản đã vá):

Mục tiêu	Phát hiện (mã 970 bắn)	Dương tính giả
`:8080` — **dính lỗi**	**16/20 = 80%**	—
`:8081` — **đã vá**	—	**0/20 = 0%**

Điểm mấu chốt: trên cả 40 lần, mã 970 bắn **khi và chỉ khi** lỗi thực sự tồn tại — **precision hoàn hảo**. Khoảng hụt 20% không phải lỗi phán xét mà là ngẫu nhiên ở khâu *dựng* chuỗi (trên bản dính lỗi, một chuỗi dựng đúng + phát lại token cũ *bắt buộc* phải lộ; miss ⇒ seed đó không dựng được chuỗi hợp lệ), và có thể thu hẹp bằng cách tăng ngân sách pha bảo mật. Nói cách khác: oracle **không bao giờ báo nhầm**; recall là một núm chỉnh kỹ thuật.

5. Trả lời gọn ba câu hỏi phản biện

Câu hỏi	Trả lời (có bằng chứng)
Vấn đề có đáng làm không?	Lớp lỗ hổng API nghiêm trọng nhất theo OWASP (API1 BOLA, API5 BFLA); tiểu lớp có-trạng-thái là nguy hiểm và ít được kiểm nhất (§1).
Đã có ai làm chưa?	Chưa. Mọi công cụ gần nhất (baseline, BACScan, ARMeta, BACFuzz, AuthREST) đều khác-người/một-thời-điểm hoặc thuần chức năng; không cái nào phát lại token cũ qua một bước hạ quyền (§2).
Có chứng minh được là hữu ích?	Có — bằng thực nghiệm: SOTA mù (tái lập được), oracle của luận văn bắt đúng với precision hoàn hảo trên 40 lần chạy (§4).

6. Nguồn & trạng thái xác minh

Tất cả nguồn học thuật dưới đây đã được mở và đối chiếu **first-party** (ngày 31/07/2026).

1. [1] A. Arcuri và cộng sự (2024). *Tool report: EvoMaster—black and white box search-based fuzzing for REST, GraphQL and RPC APIs.* Automated Software Engineering, doi:10.1007/s10515-024-00478-1. ✓ **verified**
2. [2] Ö. Şahin, M. Zhang, A. Arcuri (2026). *Enhancing REST API Fuzzing with Access Policy Violation Checks and Injection Attacks.* arXiv:2604.00702. **(Đường cơ sở trực tiếp.) ✓ verified — đọc trọn PDF**
3. [4] F. Liu, Y. Zhang và cộng sự (2025). *BACScan: Automatic Black-Box Detection of Broken-Access-Control Vulnerabilities in Web Applications.* ACM CCS 2025, doi:10.1145/3719027.3744825. ✓ **verified — trang tựa + abstract**
4. [5] S. Khan và cộng sự (2026). *Multi-Agent LLM-based Metamorphic Testing for REST APIs* (công cụ ARMeta). arXiv:2605.28321. ✓ **verified — thân bài PDF (mọi MR thuần chức năng)**
5. I P. A. Dharmaadi và cộng sự (2025). *BACFuzz: Exposing the Silence on Broken Access Control Vulnerabilities in Web Applications.* arXiv:2507.15984. ✓ **verified — đọc thân bài**
6. AuthREST (2025). *Automated Testing of Broken Authentication Vulnerabilities in Web APIs.* arXiv:2509.10320. ✓ **verified — abstract**
7. [7] OWASP API Security Top 10 (2023): API1 — Broken Object Level Authorization; API5 — Broken Function Level Authorization. owasp.org/API-Security/2023. ✓ **verified**

Ghi chú: số liệu thực nghiệm ở §4 lấy từ artifact nội bộ `infra/em-results/tier1-trials/` (40 lần chạy có kiểm soát) và `infra/em-results/reports-baseline/`.